模拟太阳帆飞行器轨道演化的动力学建模

物理背景与问题

本代码模拟的是一颗携带太阳帆的航天器在太阳引力及非引力作用（包括太阳辐射压和Yarkovsky效应）下的轨道长期演化。该模型使用轨道根数（半长轴 (a)、偏心率 (e)、轨道倾角 (i)、升交点赤经 (\Omega)、近地点幅角 (\omega)、平近点角 (M)）来描述轨道变化。

非引力加速度建模

1. **太阳辐射压（Solar Radiation Pressure, SRP）**：

太阳帆受到的SRP加速度为：

$$\vec{a}_{\mathrm{SRP}} = \frac{(1+\rho) L_{\odot} A_{\mathrm{sail}}}{4\pi c r^2 m} \cdot \hat{r}$$

其中：

- ρ：反射率
- $L_{\odot}$：太阳光度
- A_{sail}：太阳帆面积
- c：光速
- r：飞行器距太阳的距离
- m：飞行器质量

2. **Yarkovsky效应**（近似模拟）：

假设航天器非对称热辐射导致一个沿轨道方向的微小推力，其加速度近似为：

$$\vec{a}_{\mathrm{Yark}} = \frac{2}{3} \alpha \frac{L_{\odot} A_{\mathrm{th}}}{4\pi c r^2 m} (\hat{v} - \hat{r})$$

- α：热响应系数（简化常数）
- A_{th}：热辐射面面积
- $\hat{v}$：单位速度方向
- $\hat{r}$：单位径向方向

动力学建模与微分方程

采用高斯行星方程（Gauss Planetary Equations）推导非引力加速度下轨道根数的演化公式，如：

- 半长轴变化率：

$$\frac{da}{dt} = \frac{2}{n\sqrt{1-e^2}} (eS \sin f + T(1 + e\cos f))$$

- 偏心率变化率：

$$\frac{de}{dt} = \frac{\sqrt{1-e^2}}{na}(S\sin f + T(\cos f + \cos E))$$

- 轨道倾角变化率：

$$\frac{di}{dt} = \frac{r\cos(\omega + f)}{na^2\sqrt{1-e^2}}W$$

其中 S, T, W 分别为非引力加速度在径向、横向和法向方向的投影分量，f 为真近点角，E 为偏近点角。

数值实现

- 使用 `odeint` 解决上述6维常微分方程系统；
- 初始条件为几乎圆形轨道，轨道倾角1°；
- 动力学方程中每一时刻计算近点角、单位矢量和非引力加速度；
- 轨道参数以时间演化绘图；
- 最后用 `FuncAnimation` 制作二维轨道运动动画。

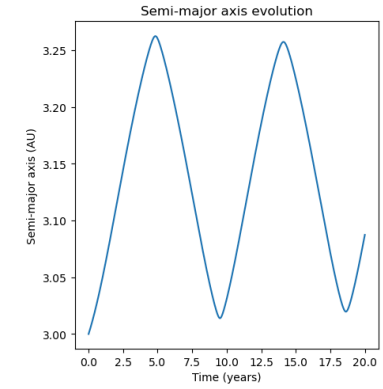
In [1]:
```
import numpy as np
import matplotlib.pyplot as plt
from matplotlib.animation import FuncAnimation
from scipy.integrate import odeint
from astropy.constants import G, M_sun

AU = 1.496e11
G_val = G.value
M_sun_val = M_sun.value
L_sun = 3.828e26
c = 2.998e8
year = 365.25 * 24 * 3600

m = 3.93
A_sail = 25
rho = 0.9
A_th = 2
alpha = 0.5

a0 = 3 * AU
e0 = 1e-6
i0 = np.radians(1.0)
Omega0 = 0.0
omega0 = 0.0
M0 = 0.0
Q0 = Omega0 * np.sin(i0)
P0 = omega0 * np.sin(i0)
y0 = [a0, e0, i0, Q0, P0, M0]

r0 = np.array([a0*(1-e0), 0, 0])
v0 = np.array([0, np.sqrt(G_val*M_sun_val*(1+e0)/(a0*(1-e0))), 0])

t_max = 20 * year
n_steps = 10000
t = np.linspace(0, t_max, n_steps)
```

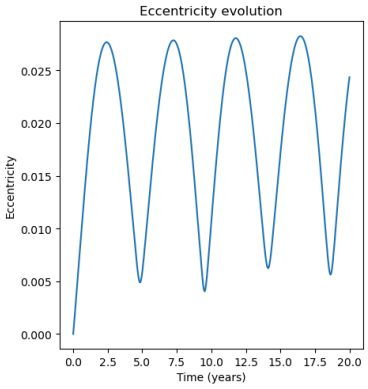
```python
def R_z(angle):
    """绕Z轴旋转矩阵"""
    c, s = np.cos(angle), np.sin(angle)
    return np.array([[c, -s, 0],
                     [s, c, 0],
                     [0, 0, 1]])

def R_x(angle):
    """绕X轴旋转矩阵"""
    c, s = np.cos(angle), np.sin(angle)
    return np.array([[1, 0, 0],
                     [0, c, -s],
                     [0, s, c]])

def solve_kepler(M, e, tolerance=1e-8):
    E = M if M < np.pi else M - e
    for _ in range(50):
        delta = (E - e * np.sin(E) - M) / (1 - e * np.cos(E))
        E -= delta
        if abs(delta) < tolerance:
            break
    return E

def orbital_elements_derivatives(y, t):
    a, e, i, Q, P, M = y

    epsilon = 1e-12
    sin_i = np.sin(i) + epsilon
    Omega = Q / sin_i if sin_i > epsilon else 0
    omega = P / sin_i if sin_i > epsilon else 0

    n = np.sqrt(G_val * M_sun_val / a**3)
    E = solve_kepler(M, e)
    f = 2 * np.arctan2(np.sqrt(1+e)*np.sin(E/2), np.sqrt(1-e)*np.cos(E/2))
    r = a * (1 - e * np.cos(E))
    p = a * (1 - e**2)

    # 坐标系转换
    r_orb = np.array([r*np.cos(f), r*np.sin(f), 0])
    v_orb = np.sqrt(G_val*M_sun_val*p)/r * np.array([-np.sin(f), e+np.cos(f), 0]
    R = R_z(Omega) @ R_x(i)
    r_ecl = R @ r_orb

    # 非引力效应
    F_rad = (1+rho) * L_sun * A_sail / (4*np.pi*c*r**2) * r_ecl/np.linalg.norm(r
    F_yark = (2/3)*alpha*(L_sun/(4*np.pi*r**2))*A_th/c * (v_orb/np.linalg.norm(v
    a_total = (F_rad + F_yark) / m

    a_orb = R.T @ a_total
    S = a_orb[0]
    T = a_orb[1]
    W = a_orb[2]

    u = omega + f
    da_dt = (2/(n*np.sqrt(1-e**2))) * (e*S*np.sin(f) + T*(1+e*np.cos(f)))
    de_dt = (np.sqrt(1-e**2)/(n*a)) * (S*np.sin(f) + T*(np.cos(f)+np.cos(E)))
    di_dt = (r*np.cos(u)/(n*a**2*np.sqrt(1-e**2))) * W

    dQ_dt = (r*np.sin(u)/(n*a**2*np.sqrt(1-e**2))) * W  # 原dΩ/dt * sin i
```

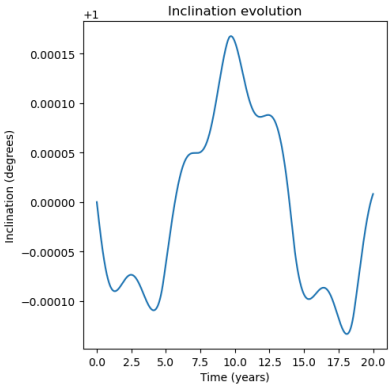
```
    dP_dt = (np.sqrt(1-e**2)/(n*a*max(e,epsilon)) * (-S*np.cos(f) + T*(1+r/p)*np
    dM_dt = n - ((1-e**2)/(n*a*max(e,epsilon))) * (-S*(np.cos(f)-2*e*r/p) + T*(1

    return [da_dt, de_dt, di_dt, dQ_dt, dP_dt, dM_dt]

# 求解轨道根数微分方程
sol = odeint(orbital_elements_derivatives, y0, t, atol=1e-12, rtol=1e-10)

a = sol[:, 0] / AU
e = sol[:, 1]
i = np.degrees(sol[:, 2])
Omega = np.degrees(sol[:, 3] / (np.sin(sol[:, 2]) + 1e-12))
omega = np.degrees(sol[:, 4] / (np.sin(sol[:, 2]) + 1e-12))
M = sol[:, 5]

plt.figure(figsize=(15, 10))

# 1. Orbital elements vs time
plt.subplot(2, 3, 1)
plt.plot(t/year, a)
plt.xlabel('Time (years)')
plt.ylabel('Semi-major axis (AU)')
plt.title('Semi-major axis evolution')

plt.subplot(2, 3, 2)
plt.plot(t/year, e)
plt.xlabel('Time (years)')
plt.ylabel('Eccentricity')
plt.title('Eccentricity evolution')

plt.subplot(2, 3, 3)
plt.plot(t/year, i)
plt.xlabel('Time (years)')
plt.ylabel('Inclination (degrees)')
plt.title('Inclination evolution')

plt.subplot(2, 3, 4)
plt.plot(t/year, Omega)
plt.xlabel('Time (years)')
plt.ylabel('RAAN (degrees)')
plt.title('Longitude of ascending node')

plt.subplot(2, 3, 5)
plt.plot(t/year, omega)
plt.xlabel('Time (years)')
plt.ylabel('AOP (degrees)')
plt.title('Argument of periapsis')

plt.subplot(2, 3, 6)
plt.plot(t/year, M)
plt.xlabel('Time (years)')
plt.ylabel('Mean anomaly (rad)')
plt.title('Mean anomaly evolution')

plt.tight_layout()
plt.show()

# 2. Orbital analysis plots
plt.figure(figsize=(12, 6))
```

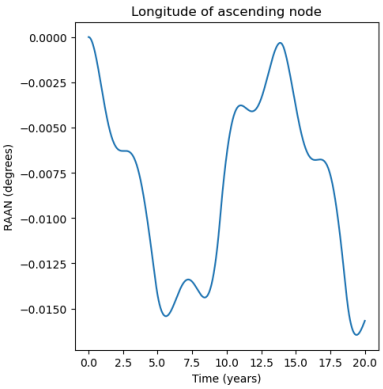
```python
# a-e diagram
plt.subplot(1, 2, 1)
plt.plot(a, e)
plt.xlabel('Semi-major axis (AU)')
plt.ylabel('Eccentricity')
plt.title('a-e orbital evolution')
plt.grid(True)

# i-a diagram
plt.subplot(1, 2, 2)
plt.plot(a, i)
plt.xlabel('Semi-major axis (AU)')
plt.ylabel('Inclination (degrees)')
plt.title('i-a orbital evolution')
plt.grid(True)

plt.tight_layout()
plt.show()

# 3. 轨道动画
fig_anim, ax_anim = plt.subplots(figsize=(8, 8))
ax_anim.set_xlim(-5, 5)
ax_anim.set_ylim(-5, 5)
ax_anim.set_aspect('equal')
ax_anim.set_xlabel('X (AU)')
ax_anim.set_ylabel('Y (AU)')
ax_anim.set_title('Spacecraft Orbital Motion')
ax_anim.grid(True)

sun = plt.Circle((0, 0), 0.1, color='yellow', label='Sun')
ax_anim.add_patch(sun)

line, = ax_anim.plot([], [], 'b-', lw=1, alpha=0.5, label='Orbit')
point, = ax_anim.plot([], [], 'ro', markersize=8, label='Spacecraft')
time_text = ax_anim.text(0.02, 0.95, '', transform=ax_anim.transAxes, fontsize=1
ax_anim.legend(loc='upper right')

def get_position(a, e, i, Q, P, M):
    epsilon = 1e-12
    sin_i = np.sin(i) + epsilon

    Omega = Q / sin_i
    omega = P / sin_i

    E = solve_kepler(M, e, tolerance=1e-8)
    f = 2 * np.arctan2(np.sqrt(1 + e) * np.sin(E / 2), np.sqrt(1 - e) * np.cos(E

    r = a * (1 - e * np.cos(E))
    x_orb = r * np.cos(f)
    y_orb = r * np.sin(f)

    R = R_z(Omega) @ R_x(i)
    pos_ecl = R @ np.array([x_orb, y_orb, 0])

    return pos_ecl[0]/AU, pos_ecl[1]/AU

step = max(1, len(t) // 1000)
frames = range(0, len(t), step)
x_pos = []
y_pos = []
```

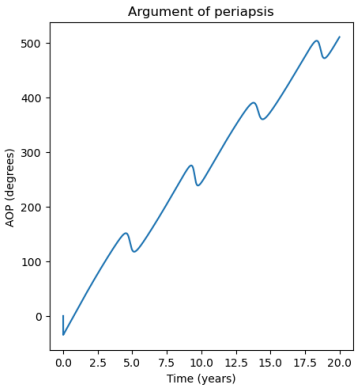
```
for j in frames:
    x, y = get_position(
        sol[j, 0],  # a in meters
        sol[j, 1],  # e
        sol[j, 2],  # i
        sol[j, 3],  # Omega
        sol[j, 4],  # omega
        sol[j, 5]   # M
    )
    x_pos.append(x)
    y_pos.append(y)

x_pos = np.array(x_pos)
y_pos = np.array(y_pos)

def init():
    line.set_data([], [])
    point.set_data([], [])
    time_text.set_text('')
    return line, point, time_text


def update(frame_idx):
    frame = frames[frame_idx]

    point.set_data([x_pos[frame_idx]], [y_pos[frame_idx]])

    line.set_data(x_pos[:frame_idx+1], y_pos[:frame_idx+1])

    time_text.set_text(f'Time: {t[frame]/year:.2f} years')

    current_x, current_y = x_pos[frame_idx], y_pos[frame_idx]
    xlim = ax_anim.get_xlim()
    ylim = ax_anim.get_ylim()

    if (current_x < xlim[0] + 0.1*(xlim[1]-xlim[0]) or
        current_x > xlim[1] - 0.1*(xlim[1]-xlim[0]) or
        current_y < ylim[0] + 0.1*(ylim[1]-ylim[0]) or
        current_y > ylim[1] - 0.1*(ylim[1]-ylim[0])):

        x_min, x_max = min(x_pos[:frame_idx+1]), max(x_pos[:frame_idx+1])
        y_min, y_max = min(y_pos[:frame_idx+1]), max(y_pos[:frame_idx+1])
        margin = 0.5
        ax_anim.set_xlim(min(-3, x_min - margin), max(3, x_max + margin))
        ax_anim.set_ylim(min(-3, y_min - margin), max(3, y_max + margin))

    return line, point, time_text

ani = FuncAnimation(fig_anim, update, frames=len(frames),
                    init_func=init, blit=True, interval=50)

plt.tight_layout()
plt.show()
```

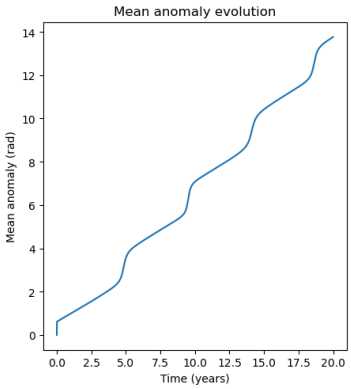

Semi-major axis evolution
Semi-major axis (AU)
Time (years)

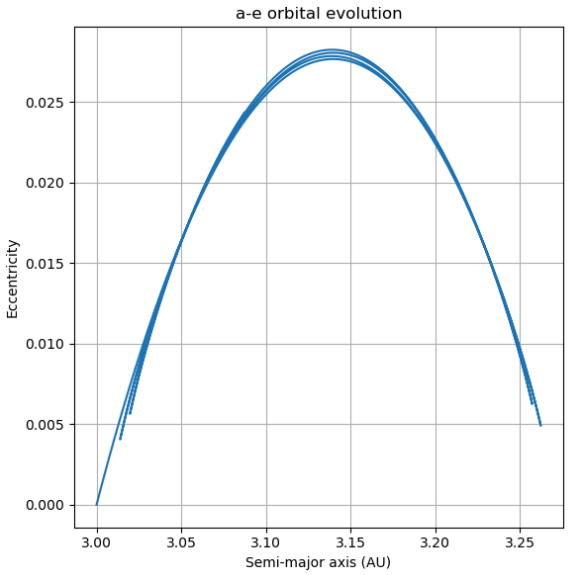

Eccentricity evolution
Eccentricity
Time (years)

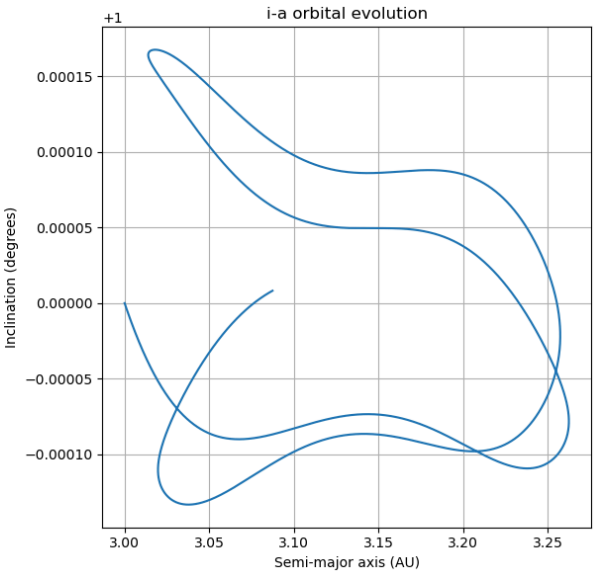

Inclination evolution
+1
Inclination (degrees)
Time (years)

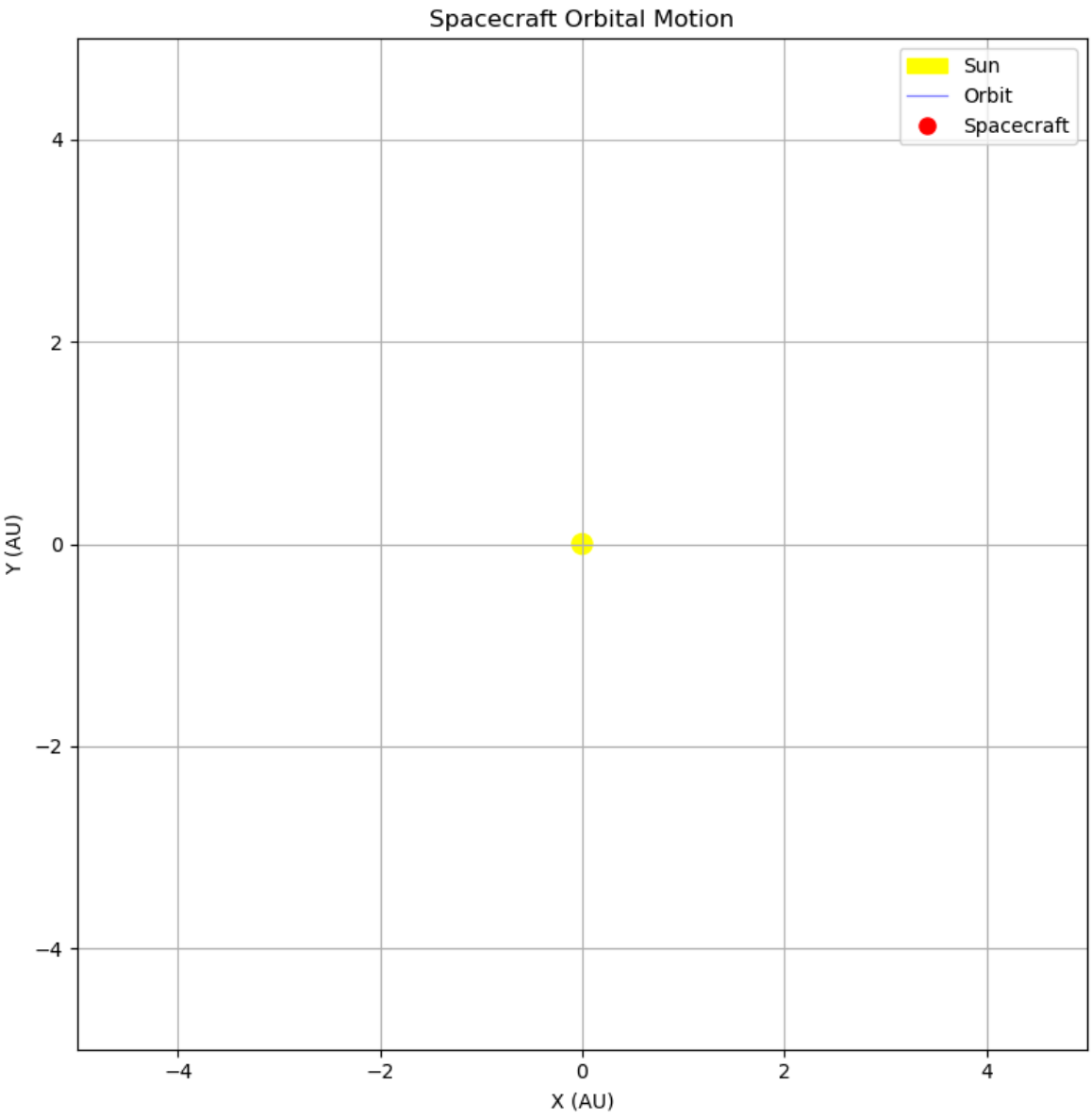

Longitude of ascending node
RAAN (degrees)
Time (years)


Argument of periapsis
AOP (degrees)
Time (years)


Mean anomaly evolution
Mean anomaly (rad)
Time (years)


a-e orbital evolution
Eccentricity
Semi-major axis (AU)


i-a orbital evolution
+1
Inclination (degrees)
Semi-major axis (AU)


Spacecraft Orbital Motion
Sun
Orbit
Spacecraft
4
2
0
−2
−4
Y (AU)
−4
−2
0
2
4
X (AU)